Assignment 01 Full Stack

(Priyanshu Sharma B.tech (cse) 2nd Year (B)

Folder Structure for a Full Stack E-Commerce Application

A full-stack e-commerce application can be divided into three main parts: the React frontend, the Express backend, and the MySQL database. Keeping these parts organized makes the project easy to understand, develop, test, and maintain. ecommerce-app/

```
■
■■■ frontend/                # React Frontend
■  ■■■ public/
■  ■■■ src/
■     ■■■ components/         # Reusable UI components
■     ■■■ pages/             # Home, Login, Products, Cart
■     ■■■ services/           # API calls
■     ■■■ context/           # Global state
■     ■■■ hooks/             # Custom React hooks
■     ■■■ assets/            # Images, icons, CSS
■     ■■■ App.jsx
■     ■■■ main.jsx
■
■■■ backend/                 # Express Backend
■  ■■■ controllers/           # Application/business logic
■  ■■■ routes/               # API routes
■  ■■■ models/                # Database models/queries
■  ■■■ middleware/            # Auth, errors, etc.
■  ■■■ config/               # Database configuration
■  ■■■ services/              # Backend services
■  ■■■ app.js
■  ■■■ server.js ■
■■■ database/
■  ■■■ schema.sql             # Tables and structure
■  ■■■ seed.sql              # Initial/sample data ■
■■■ .env                    # Environment variables
■■■ .gitignore
■■■ package.json
■■■ README.md
```

Why should routes and models not import each other directly?

Routes should mainly receive HTTP requests and send them to controllers, while models should mainly handle database operations. Keeping these layers separate reduces tight coupling and makes the application easier to test, maintain, and change.

1. Hardcoded Localhost URL and Environment Variable

The given React component uses **http://localhost:5000**. This address normally points to the computer on which the browser is running. During development this can work because the backend may be running on the same computer. After deployment, the backend will normally have a real server address or domain, so the hardcoded localhost address will not point to the production backend.

Original fetch line:

fetch("http://localhost:5000/api/products") **Use an**

environment variable with Vite:

```
# .env
VITE_API_URL=https://api.example.com

# React
```

fetch(`${import.meta.env.VITE_API_URL}/api/products`) **Complete example:**

```
function ProductList() {   useEffect(() => {    fetch(`$
{import.meta.env.VITE_API_URL}/api/products`)      .then(res => res.json())
    .then(setProducts);
 }, []);

 // ...
}
```

2. .env File and dotenv Loading in Node.js/Express

Environment variables are used to store configuration values that may change between development and production. For this backend, PORT controls the server port, DB_URL contains the database connection information, and JWT_SECRET is used for JWT-related security.

.env file:

```
PORT=5000
DB_URL=mysql://root:password@localhost:3306/ecommerce
JWT_SECRET=my_super_secret_key
```

Install dotenv: npm install dotenv

Load the variables in Express:

```
const express = require('express'); const dotenv =
require('dotenv');

dotenv.config();

const app = express();

const PORT = process.env.PORT; const DB_URL =
process.env.DB_URL; const JWT_SECRET =
process.env.JWT_SECRET;

app.listen(PORT, () => {   console.log(`Server running on port $
{PORT}`); });
```

Why should .env never be committed to Git? Because it can contain passwords, database credentials, API keys, and secret values that should not be publicly exposed.

3. Fixing a CORS Error in Express

CORS stands for Cross-Origin Resource Sharing. When a React frontend and Express backend run on different origins (for example, different ports or domains), the browser applies its same-origin security rules.
The backend must explicitly allow the frontend origin when cross-origin requests are intended.

Install the CORS package: npm

```
install cors
```

Express middleware:

```
const express = require('express'); const cors =
require('cors');
const app = express();

app.use(cors());

app.use(express.json());
```

```
app.get('/api/products', (req, res) => {   res.json({ message:
'Products loaded' }); });

app.listen(5000);
```

Why does the browser block the request? The browser blocks or restricts a cross-origin request when the server response does not provide the required CORS permission headers.

4. REST API Contract for a Book Library System

A REST API provides a standard way for the frontend to communicate with the backend. The four basic operations are Create, Read, Update, and Delete (CRUD). The following contract uses a simple Book Library example.

Endpoint	Method	Request Body	Response
/api/books	POST	{"title":"Java Basics","author":"A. Kumar"	} 201: {"message":"Book created","book": {...}}
/api/books/:id	GET	None	200: {"id":1,"title":"Java Basics","author":"A. Kumar"
/api/books/:id	PUT	{"title":"Advanced Java","author":"A. Ku	mar"200: {} "message":"Book updated","book":{...}
/api/books/:id	DELETE	None	200: {"message":"Book deleted"}

}

}

Explanation: POST is used to create a new book. GET reads book information. PUT updates an existing book. DELETE removes a book. The response normally contains a status code and JSON data.

5. Reusing One Database Connection

The original code creates a new MySQL connection every time the /products route is called. For a real application, repeatedly opening database connections is inefficient and can become a performance problem when many users send requests at the same time.

Improved code using a reusable connection pool:

```
const express = require('express'); const mysql =
require('mysql2');

const app = express();

const db = mysql.createPool({   host:
'localhost',   user: 'root',   password: '',
database: 'ecommerce',
waitForConnections: true,
connectionLimit: 10,   queueLimit: 0 });

app.get('/products', (req, res) => {   db.query('SELECT * FROM products', (err,
rows) => {    if (err) {      return res.status(500).json({ error: err.message });
  }
  res.json(rows);   }); });

app.listen(5000);
```

Why is the original version a performance problem? Opening a new connection for every request adds

connection setup overhead and can consume database resources quickly. A connection pool creates a limited number of reusable connections and shares them between requests.

6. package.json Scripts Using concurrently

The concurrently package allows multiple commands to run at the same time in one terminal. Here, the backend runs with nodemon and the React frontend runs inside the client folder.

Install required packages: npm install --save-dev

concurrently nodemon **scripts section of**

package.json:

```
{
  "scripts": {
    "server": "nodemon server.js",
    "client": "npm start --prefix client",
    "dev": "concurrently \"npm run server\" \"npm run client\""
  }
}
```

Now, running **npm run dev** starts both the Express backend and the React frontend together. This saves time because the developer does not need to open two separate terminals for normal development.

7. API Key Inside a React Component

The teammate's code places an API key directly inside frontend source code. This is unsafe because React code is sent to the browser and can be inspected by users. Therefore, a secret key should not be treated as private merely because it is stored in a frontend environment variable.

Development example using a frontend environment variable:

```
# .env
VITE_WEATHER_API_KEY=your_key_here

# React
const response = await fetch(   `https://api.weather.com/data?key=${import.meta.env.VITE_WEATHER_API_KEY}`
);
```

Important security point: Vite variables beginning with VITE_ are exposed to the frontend bundle. They are useful for non-secret configuration, but they do not make a secret API key secure. For a truly secret API key, the request should normally be made by the backend, where the secret can remain on the server.

8. Full Login Request-Response Cycle

When a user submits a login form, the request travels from the React UI to the Express backend. The backend validates the data, checks the database, and sends a response. The frontend then updates the UI according to the result.

```
// Step 1: User enters username and password in the React login form.

// Step 2: User clicks the Login button.
// Step 3: React form handler prevents the normal page refresh.

// Step 4: React creates a POST request such as:
//       POST / api/login

//       Body: { username, password } // Step 5: Express

receives the request.
```

```
// Step 6: express.json() reads the JSON request body, //        making the data
available through req.body.

// Step 7: The login route sends the username to the //        appropriate
controller/service.

// Step 8: The backend queries MySQL to find the user.

// Step 9: The backend checks the submitted password //        against the
stored password information.

// Step 10: If the credentials are valid, the backend //        creates the appropriate login/
session/JWT response.

// Step 11: Express sends a success response such as:

//        { message: "Login successful", token: "..." } // Step 12: React receives the

response.

// Step 13: React updates application state and/or stores //        the required
authentication information.

// Step 14: The UI changes to show the logged-in user, //        dashboard, or
another protected page.

// If credentials are invalid, the backend sends an error // response and React displays
a suitable login error message.
```

9. Fixing req.body Undefined in Express

The problem is that Express does not automatically parse a JSON request body unless the appropriate body-parsing middleware is enabled. In modern Express applications, **express.json ()** is used to parse JSON request bodies and make the data available in **req.body**.

Corrected code:

```
const express = require('express');

const app = express();

// Missing line: enable JSON body parsing app.use(express.json());

app.post('/login', (req, res) => {   const { username,
password } = req.body;

  res.send(`Welcome ${username}`); });

app.listen(3000, () => {
  console.log('Server running on port 3000'); });
```

Missing line: app.use(express.json());

This middleware reads JSON sent by the client and converts it into a JavaScript object. Therefore, the route can safely access values such as req.body.username and req.body.password.